Aegis deployment to existing OCI resources

Final deployment handover for Swamy
Windows PowerShell | Branch ags_sprint1 | 16 October 2026

Use this guide to deploy the selected Oracle database objects, frontend and backend code. Choose either the AI skill or JSON plus PowerShell. Both use the maintained repository tools and preserve existing infrastructure, application settings, stored data and model registrations.

Two supported ways to deploy

Route	What Swamy does	Page
AI skill	Give the target inputs to aegis-deploy; approve discovery and then the complete release plan.	6
JSON + PowerShell	Fill one target JSON file and run deploy.ps1 in an interactive terminal.	7

One terminal command from the repository root:

```
.\ags_deploy\deploy.ps1 -Inputs C:\aegis-target\target.json
```

Scope: DB_Object Y creates only listed missing objects, and a missing application user/schema when the release file permits it. DeployFrontend Y updates web. DeployBackend Y updates API and worker together. Each component can be selected separately.

Prepared OCI is required. The physical Autonomous AI Database, OKE workloads, storage, Secrets, saved connections, model assets and routing must already be prepared. This command cannot turn an empty tenancy or bare cluster into a working installation. Missing initial setup must be completed separately by the platform/application owner.

Existing configuration is preserved. No infrastructure provisioning, runtime configuration changes, model downloads, existing user/grant changes, data import or automatic rollback are part of this release. No backup is created or required by this workflow. Actual image updates can briefly interrupt service with the existing Recreate strategy.

Read in this order

2 Prepare the destination and tools
3 Obtain source and fill target inputs
4 Choose components and database scripts
5 Understand database safeguards and runtime profile
6 Use the AI skill **or** 7 Run PowerShell
8 Verify the result and handle a stopped release

1 Confirm the destination is prepared

For another tenancy or database, obtain that destination's access and configuration. Do not reuse the original live wallet, owner, kubeconfig or endpoint. Tenant details alone are insufficient. Ask the platform/application owner to confirm the items below before publication.

Prerequisite	Required handoff
Oracle database	An existing Autonomous AI Database supporting the maintained native VECTOR, SQL BOOLEAN and collation definitions; reachable with the target wallet and authorized account.
Application workloads	Existing Aegis Deployments and container names, namespace, Services, persistent claims and shared API/worker state. Ready nodes alone are insufficient.
Saved configuration	Target DB connection and wallet, Object Storage route, encryption key, credentials and runtime settings already installed. Existing model registrations and CPU reranker assets must be available.
Access and registry	Destination kubeconfig/context and its authentication executable; authorized Kubernetes access; existing registry repositories, push login and cluster pull access.
Public access	Prepared HTTPS routing and hostname. The tool can discover an existing URL; it does not create DNS, certificates, Ingress or a load balancer.

For a missing application schema, the release plan may label app readiness as conditional. After approved DDL, the saved connection and application must pass checks **before images are published**. Creating tables alone does not seed application users, models, documents or configuration.

Prepare the release computer

Use Windows PowerShell 5.1 or newer, Python 3.10 or newer (3.12 recommended), and Git. DB object creation needs the Oracle Python driver. Application releases additionally need Docker in Linux-container mode, kubectl, registry login, and the authentication executable named by the destination kubeconfig. Standard OCI CLI kubeconfigs require OCI CLI and its existing identity.

Keep the wallet, kubeconfig, private keys and target JSON outside Git. The current Mumbai pilot kubeconfig uses a private local authentication helper/state; it is not portable to another tenancy. Obtain a destination-specific kubeconfig from its owner.

Database-only selection: no Docker, Kubernetes, registry or application URL is required. Use DB_Object Y, DeployFrontend N and DeployBackend N. Missing prerequisites stop the release; resolve them outside this code/object-only scope.

2 Prepare source and target inputs

Obtain GitHub access to the repository. On a new release computer, run these commands individually and stop on errors. If you already have a checkout or virtual environment, reuse it without overwriting pending work.

```
git clone --branch ags_sprint1 https://github.com/Maniatramco/Aegis.git
Set-Location .\Aegis
py -3.12 -m venv .venv
.\.venv\Scripts\python.exe -m pip install `
  -r ags_backend\requirements-oracle.txt
New-Item -ItemType Directory -Path C:\aegis-target -Force
Copy-Item .\ags_deploy\deployment-inputs.example.json `
  C:\aegis-target\target.json
git rev-parse HEAD
```

Fill the copied file with known destination values. Blank supported fields are prompted or discovered after permission. Ref may be an exact tested commit; ags_sprint1 selects that branch's resolved commit. The release builds an isolated Git export of that commit, excluding uncommitted and untracked files.

JSON fields	What to supply
SourcePath / Ref	Absolute local repository path; tested branch, tag or commit. Record the exact commit shown in the plan.
Kubeconfig / Context / Namespace	Destination kubeconfig path, its exact context, and existing Aegis namespace.
Registry / RegistryNamespace	OCIR hostname without https://, and tenancy registry namespace. This is different from the Kubernetes namespace.
Platform / AppUrl	linux/arm64 or linux/amd64 matching nodes. Existing HTTPS URL; leave blank to discover prepared routing.
WalletPath / DatabaseOwner / DBAdminUser	Destination wallet ZIP, application schema and existing inspection/admin account. New-schema creation needs account creation and grant privileges.
ServiceAlias / ExpectedService	Wallet TNS alias and full database SERVICE_NAME from the target descriptor. They must identify the intended database.
ValidationReference	Truthful test/CI evidence for the exact source. The deployment command records this reference; it does not execute the regression suite.

The template contains the three Y/N selectors and DB_ReleaseFile as shown on page 4. Do not add passwords, tokens, private keys or wallet contents to JSON. If using the separately filled Mumbai JSON, remember it targets the **existing live installation**; replace its destination values for another OCI environment.

3 Select exactly what to deploy

Use the following selection fields inside the full target JSON. The remaining destination fields come from the template on page 3. Use quoted Y or N values.

```
{
  "DB_Object": "Y",
  "DB_ReleaseFile": "ags_database/release.json",
  "DeployFrontend": "Y",
  "DeployBackend": "Y"
}
```

Release selection	DB_Object	Frontend	Backend
All three	Y	Y	Y
Database only	Y	N	N
Frontend only	N	Y	N
Backend API + worker only	N	N	Y
Both code components; no DB DDL	N	Y	Y

Backend includes both API and worker. Unselected images are not built, pushed or changed. All three N is rejected. With DB_Object N, a backend release still checks schema compatibility and active jobs read-only; frontend-only with DB_Object N makes no Oracle queries.

The database release file controls the script list

`ags_database/release.json` contains **CreateSchemaIfMissing: "Y"** and an ordered **Scripts** array. The supplied manifest lists 13 maintained SQL files under ags_database/ags_tables, containing 13 table and 20 explicit index definitions. Only these listed files participate; the tool does not scan a folder for more scripts.

A repository-relative manifest and all SQL paths are resolved from the selected committed source. An absolute external manifest is also supported: its content is frozen in the private record, while its SQL paths still refer to that committed source. Edit and commit repository SQL/manifest changes before selecting their release ref.

This release supports the maintained CREATE TABLE/INDEX definitions. Arbitrary migrations, stored-program releases, ALTER/DROP statements and DML scripts require separate implementation and review. Do not add them expecting a general SQL runner.

4 Review database and runtime behavior

When DB_Object is Y

Read-only preflight compares the listed object definitions against the intended owner and full service. Matching tables and indexes are skipped. Missing supported objects are created in manifest order only after approval. Incompatible definitions stop the release before DDL; row counts or table counts alone are not a schema comparison.

If the application user/schema is missing and CreateSchemaIfMissing is Y, the plan shows the new account, CREATE SESSION and CREATE TABLE grants, existing DATA tablespace and default 256 MB quota. Optional SchemaTablespace and SchemaQuotaMB select other reviewed existing-space settings. Existing accounts, passwords, grants and quotas are preserved. No physical database or tablespace is created.

Wallet, admin and application-owner passwords are entered privately when requested in the interactive terminal. A new schema password is confirmed privately. They are not saved in JSON, argv, generated SQL or release records. The database applies its password policy.

No existing application rows are imported or changed by this object release. A missing schema is an empty application schema; it does not migrate the original workspace. Database-only with every listed object already matching finishes as verified-only without PUBLISH because there is no change to apply.

Use the committed live-matching build profile

Area	Maintained release profile
Backend base	Python 3.12.15 with the immutable base digest matched to the observed live ARM64 image.
Frontend base	Node.js 22.23.3 with the immutable live-matching base digest; production npm ci uses package-lock.json.
Backend dependencies	Committed pinned requirements; Oracle, OCI and native AWS/Google SDK bundles enabled. Local in-process embeddings disabled, matching live.
Model and application state	Existing provider mappings, remote Ollama endpoints, reranker files, encryption keys, runtime settings and storage routes preserved.

Do not change Docker build flags, base digests, dependencies or model configuration during a release to work around a failed preflight. Keep required providers and any deliberately selected laptop model relay available. A powered-off laptop cannot serve its Ollama model; Gemini-mapped datasets use their existing cloud provider.

The main entry point is deploy.ps1. The application image helper, database installer and read-only guard remain implementation/support tools. Do not bypass the approval workflow by running an underlying publish helper directly.

5 Deploy using the AI skill

Give the AI your source location/ref, target JSON location and component selection. Use the repository skill at `.agents/skills/aegis-deploy/SKILL.md`. If the tool recognizes installed skills, invoke **$aegis-deploy**; otherwise ask it to read and follow that file.

Copy this request to the AI

Use $aegis-deploy and follow .agents/skills/aegis-deploy/SKILL.md in my Aegis checkout. Deploy the tested ags_sprint1 source using C:\aegis-target\target.json. Use its DB_Object, DB_ReleaseFile, DeployFrontend and DeployBackend selections. Preserve all existing OCI infrastructure, application configuration, model registrations and data. Do not provision infrastructure or change settings. Ask me for missing inputs; if unknown, request permission for bounded read-only discovery. Before publishing, show the complete verified release plan and wait for my explicit approval.

The AI must complete these stages

1. Resolve inputs locally. Check source, destination file and selected components. Ask about unknown values. Sharing a tenancy ID does not select a unique cluster or database, and the AI must not guess the target.

2. Obtain connection permission. Confirm the account/profile, tenancy, region and discovery scope before OCI, Kubernetes, Oracle or public-target checks. Existing consent for the same release scope can be reused. Discovery is read-only and does not permit publication.

3. Prepare a concrete release plan. Verify target identity and prerequisites; complete applicable source checks and local image builds. Show the exact commit, cluster/context/server, namespace, architecture, registry, URL, current and proposed images, DB owner/full service, script hashes, missing objects, any new schema/grants/quota, active jobs, preserved configuration fingerprints, evidence, unrun checks and expected restart window.

4. Obtain final approval. Wait for explicit approval of that exact plan before DDL, image pushes or workload updates. Credential access, read-only consent and successful preflight are not publication approval. Recheck changed source/target details and obtain amended approval if the plan changes.

5. Execute and report. Apply only approved missing DB objects and selected images, verify configuration and readiness, and return source/target, DB outcome, image digests, HTTPS URL, exact receipt and any limitations. Stop dependent work on failure; do not automatically retry or roll back.

Use a private credential mechanism or the operator terminal for secrets. Do not paste passwords or tokens into the AI request. An unknown application URL may be discovered from existing routing after permission; the skill must stop if routing or other initial setup is missing.

6 Deploy using JSON and PowerShell

Open an interactive PowerShell terminal at the Aegis repository root. This route runs directly without an AI agent. Complete target input preparation on pages 2-4. For an application release, ensure Docker is running and your existing destination registry login is valid.

If registry login is needed, run the command below using the destination registry host. Use Docker's interactive password prompt for the approved registry token; never add it to a command or JSON. This authenticates the local client and does not publish images.

```
$taskRegistry = 'YOUR_DESTINATION_REGISTRY_HOST'
docker login $taskRegistry
```

Optional read-only preflight

```
.\ags_deploy\deploy.ps1 `
  -Inputs C:\aegis-target\target.json -CheckOnly
```

Confirm the displayed target and type **CONNECT** only when read-only access is permitted. Supply private database passwords if prompted. CheckOnly performs remote/schema preflight without builds, DDL, image pushes or rollout. Review the printed result and resolve any prerequisite failure. It does not grant approval for a subsequent deployment.

Run the release

```
.\ags_deploy\deploy.ps1 -Inputs C:\aegis-target\target.json
```

1. Review the source and target; type **CONNECT** to authorize read-only discovery for this run. Missing non-secret inputs are prompted or discovered. Database passwords use private terminal prompts.

2. Read preflight results. If app images are selected, the command builds them locally using the exact committed source. ValidationReference records your source test/CI evidence; tests are not run by this command.

3. Review DEPLOYMENT PLAN: exact commit, target, component selection, image IDs/tags, database owner/full service, missing-object/new-schema actions, conditional readiness and preserved settings. Type **PUBLISH** only when you approve those exact changes. Declining stops publication.

4. The command rechecks source, target and artifacts, applies approved DB actions first, verifies the saved connection and readiness, then pushes and updates only selected images. Backend updates API and worker together; frontend follows when selected. Drift or failures stop dependent steps.

5. Keep the exact Application, Receipt and Record paths printed by this invocation. Successful app execution verifies HTTPS 200, selected images, readiness, schema and preserved configuration. Review result.json before declaring completion.

No backup reference is mandatory or prompted for. Do not use legacy DatabaseAction examples alongside DB_Object. The current Y/N template and this entry point are the supported handover workflow.

7 Verify and hand over the result

Retain the private run directory under ignored `ags_deploy/deployment-records/`. Use the exact paths printed by the run, rather than selecting a receipt by the newest filename. Keep destination details and records outside public Git history.

Record	What to confirm
Source and destination	Exact source commit, approved target/context/namespace, DB owner/full service and selected flags.
Database result	Created or skipped objects; new schema only if approved; final compatibility check. Database-only completion does not claim an application rollout.
Application result	Previous/new image digests, selected workload readiness, unchanged unselected images and protected configuration, existing HTTPS URL/status.
Functional acceptance	Check sign-in and read-only screens. A new upload, extraction or cited-chat test needs a prepared test workspace and explicit authorization for writes/inference.

If the command stops

Stop and inspect the recorded phase, completed DB objects and actual image versions. Oracle DDL can commit partially. A failed frontend rollout may follow a successful backend rollout. Do not blindly rerun, replay the fresh installer, run rollout undo, change settings, or claim a partial release succeeded. Recovery or restoration of previous compatible images requires its own approved action; no automatic rollback is provided.

Validation recorded on 16 October 2026

The checked source baseline below passed 1,047 backend tests (5 skipped), 167 frontend unit tests, frontend type checks and 69 offline deployment tests. ARM64 production images built successfully; frontend startup/assets returned HTTP 200; all 105 installed backend package versions matched live. Read-only Oracle comparison matched 13 tables, 165 columns, 49 foreign keys and 20 explicit indexes, with no missing listed objects.

Live health remained unchanged during that review. No live provider inference, upload/extraction/chat write workflow, new-tenancy installation or recovery drill was performed. These results support that source snapshot; a different target and changed code still need their own verification. A branch push is not deployment and is not a new CI run.

Verified application/tooling baseline: `3ae03a5f40fa24bd7c05f84892a9d73efddc677d`. The final handover adds documentation only. The original immutable tag aegis-freeze-2026-10-15 remains unchanged.

Maintained source and reference

Release entry point: ags_deploy/deploy.ps1
Input template: deployment-inputs.example.json
Object list: ags_database/release.json
AI workflow: aegis-deploy skill on ags_sprint1

Architecture, observed OCI resources, running technologies and the approved Mural diagram are maintained separately in Aegis_Architecture_Handover.docx and its matching PDF. This document replaces the separate database, backend and Kubernetes deployment guides.